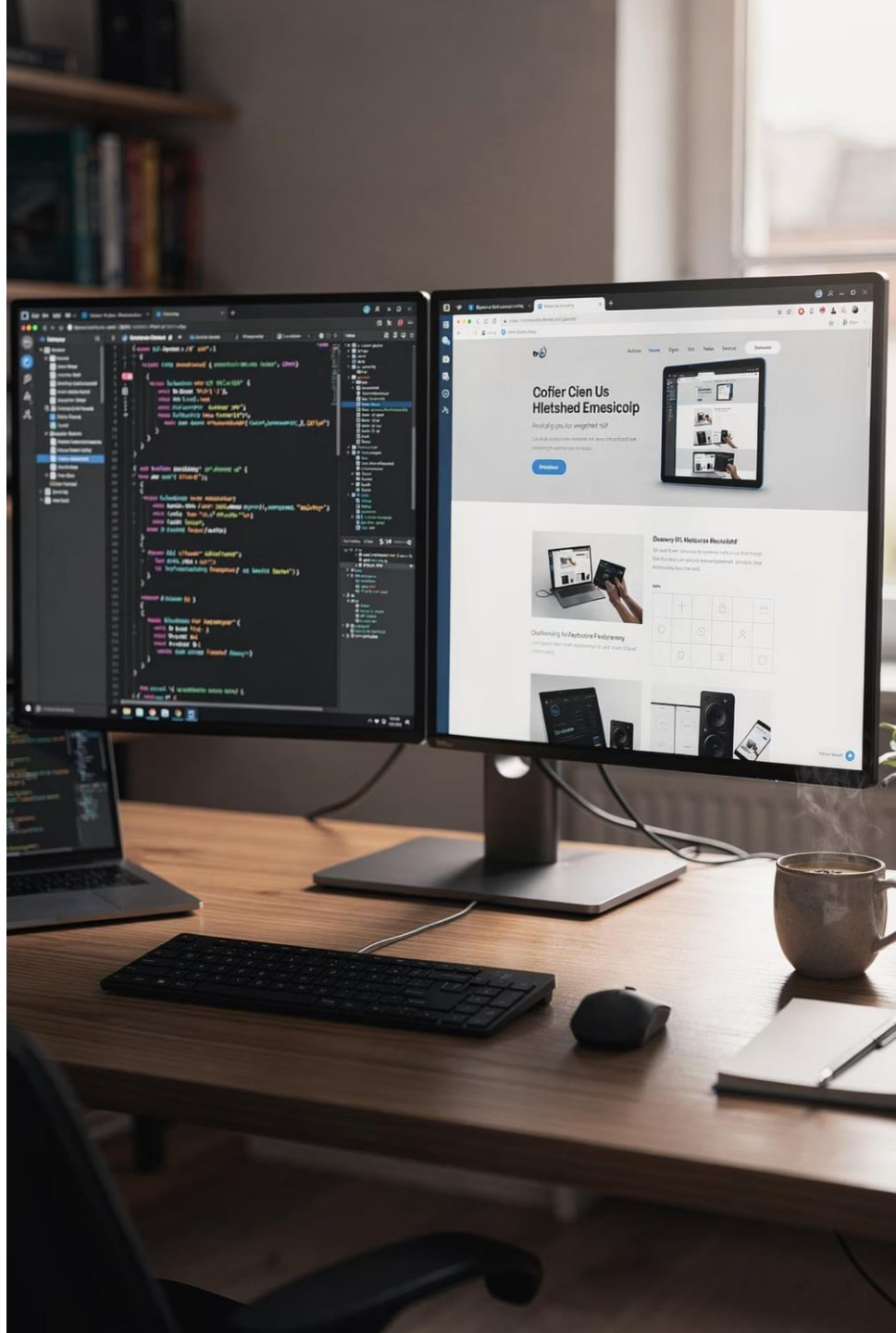
SCRUM DIGITAL
Agile Solutions For Dynamic Business Requirements

# Day 10: Managing Data with CRUD Operations

## Building Your First Complete Web Application

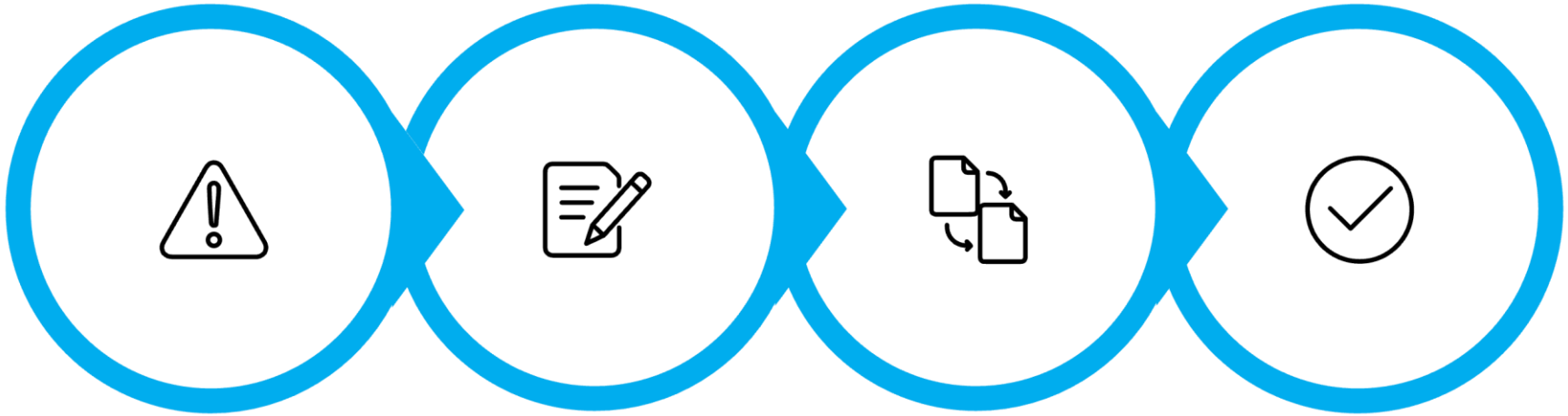
FULL STACK BOOTCAMP · DAY 10 OF 13



# 🤔 Opening Question

> Imagine you just submitted a student record — but the CGPA is wrong. Can you fix it? Can you delete it entirely?

Every real-world application needs more than just inserting data. Users make mistakes. Records become outdated. Data needs to be corrected, removed, and searched. Today, you'll build exactly that — a system that gives you full control over your data.



**Wrong Data** **Edit Clicked** **Run Update** **Record Saved**

# 🎯 Today's Mission

By the end of this session, you will have built and deployed a fully functional **Student Management System** — complete with every core CRUD operation and a polished Bootstrap UI.

**View**
Display all student records in a clean table

**Delete**
Remove records safely with a confirmation step

**Edit & Update**
Modify existing student data via a pre-filled form

**Search**
Filter records by name, branch, or email

# 📋 Session Roadmap

Every module follows the same proven four-step rhythm — so you always know what's coming next. No more than 10 minutes will pass without you writing code.

**1**

### 💡 Concept

Understand the "why" in 3 minutes

**2**

### 🧑‍💻 Live Coding

Watch the instructor build it live

**3**

### 🎯 Mission

You implement it yourself

**4**

### 🏆 Challenge

Extend it with a bonus feature

This session has 4 modules: Update · Delete · Search · Better UX — followed by a Bug Hunt, Build Sprint, and Demo Day.

# ⚡ Rapid Fire Recap

Turn to the person next to you. You have **3 minutes** to discuss these questions together. Be ready to share your answers with the class.

### 1. Purpose of MySQL

What role does MySQL play in a web application? What does it store?

### 2. INSERT vs. SELECT

What is the key difference? When do you use each one?

### 3. Primary Key

What is it, and why is it absolutely required in every table?

### 4. Form Validation

Why validate on both client-side (JS) and server-side (PHP)?

### 5. PHP 🤝 MySQL

Walk through the connection steps: credentials, connect, query, fetch.

MODULE 1

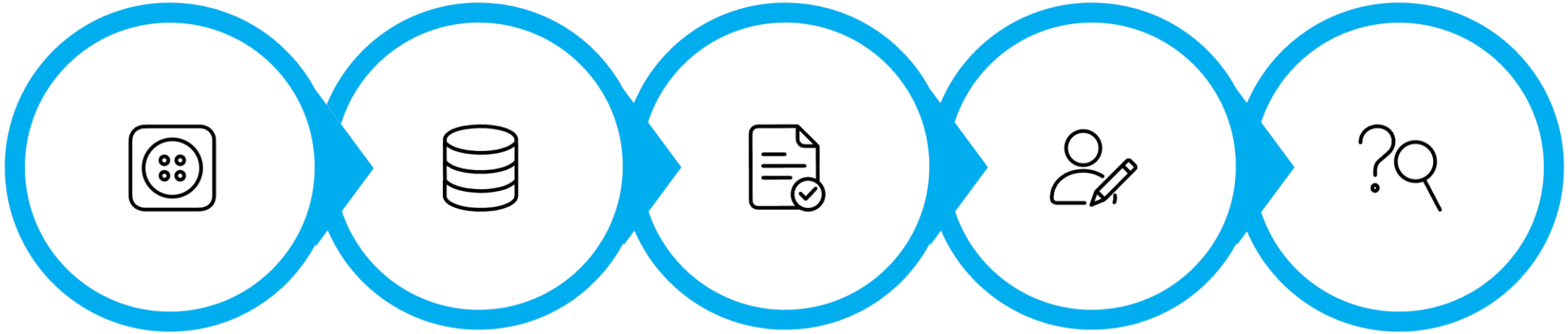
# Updating Records

# Edit. Fix. Save.

In this module you'll learn how to select a specific record, pre-populate a form with its data, and write an `UPDATE` SQL query to save the changes — all wired together through PHP.

# Module 1 · Concept: How UPDATE Works

Updating a record is a two-step process. First you **read** the record (SELECT by ID), then you **write** the changes back (UPDATE with a WHERE clause). The `WHERE` clause is critical — without it, every row in the table gets overwritten.




**The SQL Pattern**

```
UPDATE students
SET name='John', cgpa=3.8
WHERE id=5;
```

**⚠ Golden Rule**

Always include `WHERE id = ?` in your UPDATE statement. Omitting the WHERE clause will update **every single row** in your table — a mistake that is very difficult to undo.

# Module 1 · Live Coding: Edit Student

Watch carefully. The instructor will build the full Edit flow — step by step — in VS Code. You'll see the result in the browser after each step.

01

### Add Edit Button

Add an "Edit" button to each row in `students.php` that passes the student's `id` via the URL: `edit.php?id=5`

02

### Fetch the Record

In `edit.php`, use `$_GET['id']` to run a `SELECT` query and retrieve that student's current data.

03

### Pre-fill the Form

Populate each input field's `value` attribute with the fetched data so the user sees the current values.

04

### Run the UPDATE Query

On form submit, run the `UPDATE students SET ... WHERE id=?` query using prepared statements.

05

### Redirect with Message

Redirect to `students.php` and display a Bootstrap success alert: "Record updated successfully."

# Module 1 · 🎯 Your Mission + 🏆 Challenge

### 🎯 Mission — Implement Edit Student

Build the complete edit flow for your Student Management System. Your edit form must allow changes to all four fields:

- Student Name
- Email Address
- Branch / Department
- CGPA (validate: 0.0 – 10.0)

### 🏆 Challenge

Display a Bootstrap `alert-success` green banner immediately after a successful update. The message should auto-dismiss after 3 seconds using JavaScript.

### ⭐ Bonus

Add a `updated_at` TIMESTAMP column to your table and display "Last Updated: [time]" next to each student row.

MODULE 2

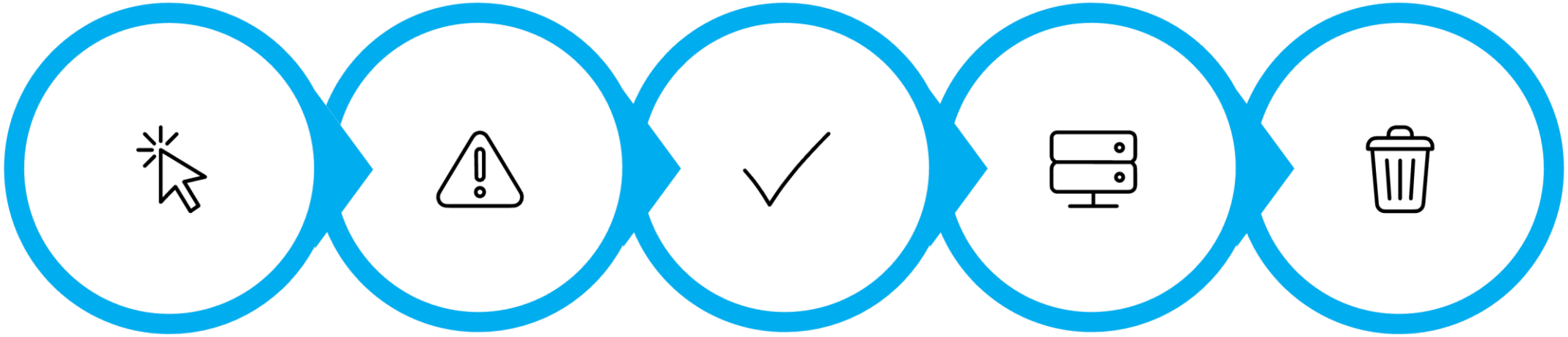
# Deleting Records

# Confirm. Then Delete.

Deleting is permanent. In this module you'll build a safe delete flow with a JavaScript confirmation dialog — because a good developer always asks "Are you sure?" before destroying data.

# Module 2 · Concept: How DELETE Works

A delete operation is simple but dangerous. One unprotected `DELETE` without a `WHERE` clause wipes an entire table. The safest pattern: confirm in JavaScript first, then execute in PHP only if confirmed.

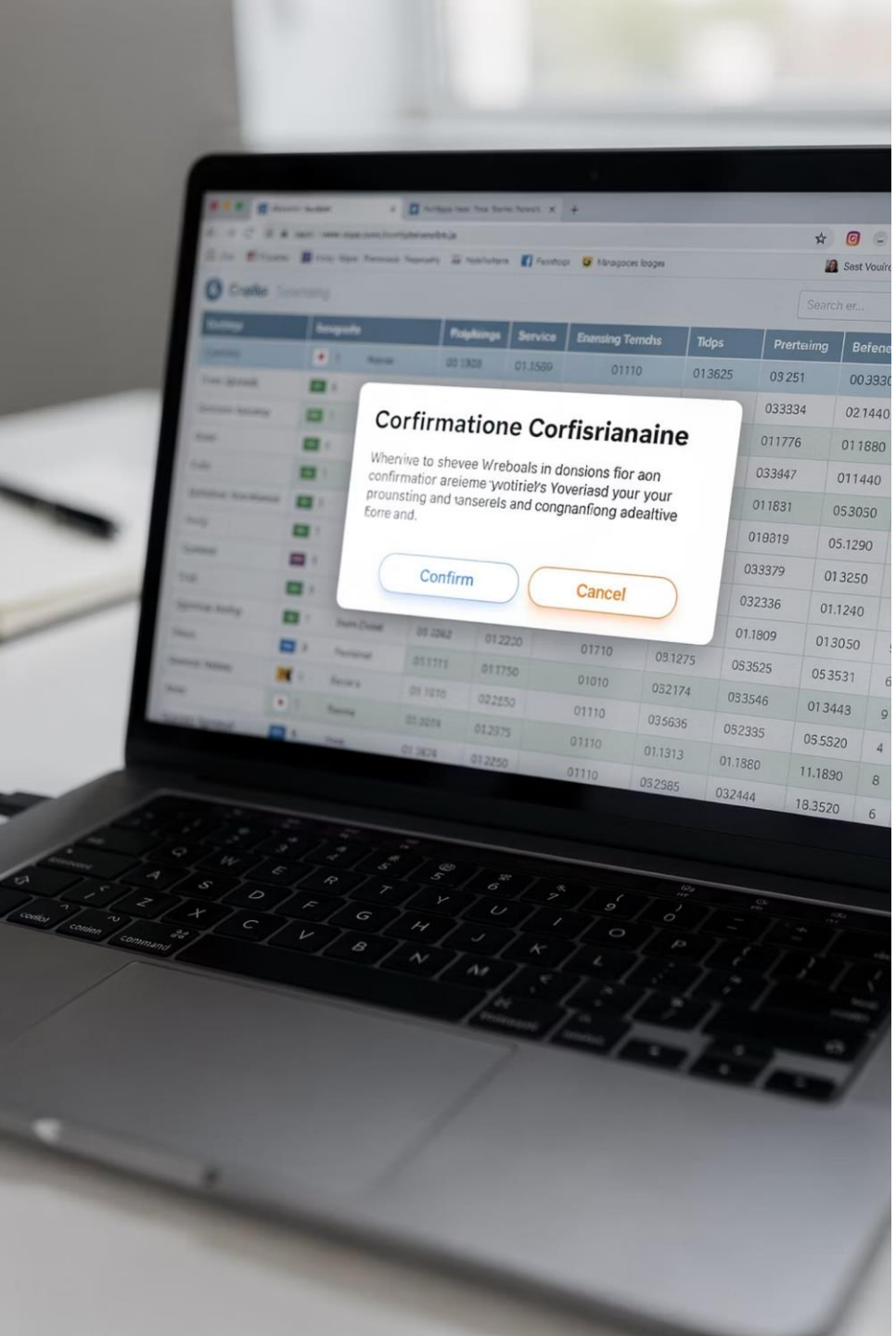



### The SQL Pattern

```
DELETE FROM students
WHERE id = 5;
```

### Why the Confirmation Dialog Matters

Accidental clicks happen. A single `window.confirm()` call costs 1 line of JavaScript and prevents irreversible data loss. In production systems, a soft-delete approach (marking records as deleted rather than removing them) is even safer.




# Module 2 · 🎯 Your Mission + 🏆 Challenge

## 🎯 Mission

Add a **Delete** button to each row in your student table. Clicking it should pass the student's ID to `delete.php`, run a `DELETE FROM students WHERE id=?` query, and redirect back to the table.

## 🏆 Challenge

Before the delete request is sent, show a JavaScript `confirm()` dialog: *"Are you sure you want to delete this record? This action cannot be undone."* Only proceed if the user clicks OK.

## ⭐ Bonus

After deletion, display a Bootstrap `alert-danger` red banner: *"Record deleted successfully."* Include the student's name in the message for extra clarity.

MODULE 3

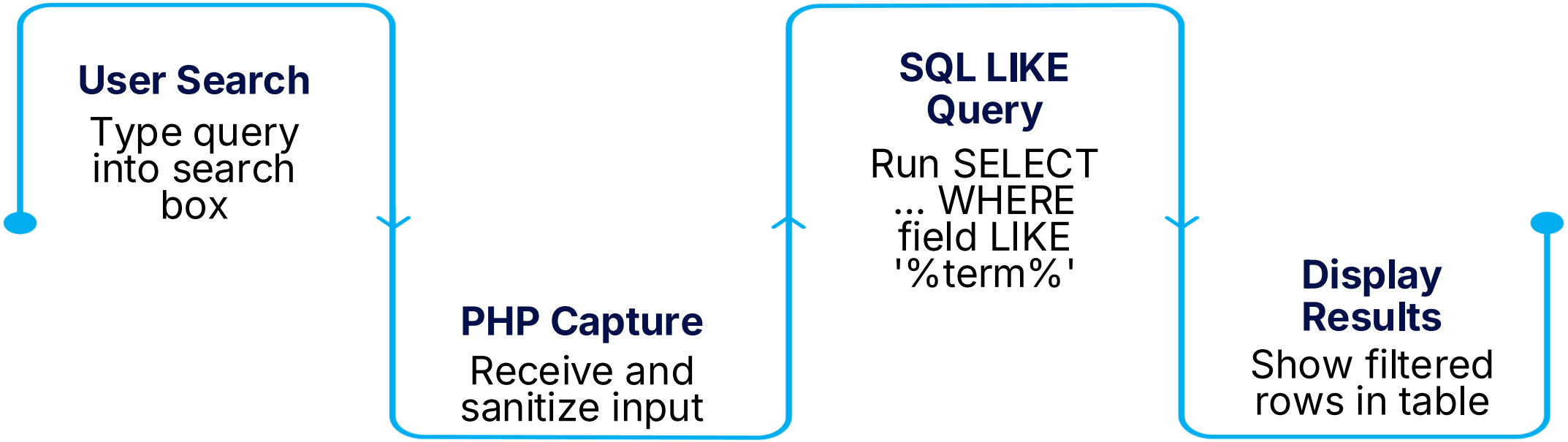
# Searching Records

# Find. Filter. Fast.

When your table has hundreds of students, scrolling is not a solution. In this module you'll build a live search feature that filters results by name, branch, or email using PHP and SQL `LIKE` queries.

# Module 3 · Concept: How Search Works

Search uses the SQL `LIKE` operator with wildcard characters. The `%` symbol matches any number of characters before or after your search term, making it flexible for partial-word matches.




**The SQL Pattern**

```
SELECT * FROM students
WHERE name LIKE '%john%'
OR email LIKE '%john%'
OR branch LIKE '%john%';
```

**Search Best Practices**

- Always sanitize the search input before using it in a query
- Use prepared statements to prevent SQL injection
- Show a "No results found" message for empty result sets
- Preserve the search term in the input box after submission

# Module 3 · 🎯 Your Mission + 🏆 Challenge

Time to wire up search in your Student Management System. Your goal is a functional search bar that filters the student table in real time on form submit.

## 1

### 🎯 Mission Requirements

- Add a Bootstrap search input above the student table
- Search by student name using a `LIKE` query
- Display only matching records in the table
- Show a "No students found" row if no matches exist

## 2

### 🏆 Challenge

Extend your search to also filter by **Branch**. Use an `OR` clause so a single search term matches name *or* branch.

## 3

### ⭐ Bonus

Use PHP's `str_replace()` or JavaScript to **highlight the matching keyword** in yellow inside the search results — just like Google does.

MODULE 4

## Better User Experience

# Polish Matters.

Features without good UX feel broken. In this final module you'll add Bootstrap alerts, empty states, hover effects, and a professional table layout — turning your working app into a *great* app.

# Module 4 · Concept: UX That Users Notice

The difference between a student project and a professional application often comes down to the small things: feedback messages, empty states, and visual polish. These take minutes to add but make a world of difference.

### Success Alerts

Bootstrap `alert-success` and `alert-danger` confirm to the user that their action worked — or failed. Never leave the user guessing.

### Professional Table

Use Bootstrap's `table-hover`, `table-striped`, and `table-bordered` classes. Add action buttons with icons using Bootstrap Icons for a polished, modern look.

### Empty State

When no records exist or no search matches, show a friendly "No students found" message. A blank table looks broken; an empty state looks intentional.

### Record Count

Display the total number of students with a Bootstrap badge: *"Showing 12 students."* Small details like this build trust in the application.

# Module 4 · 🎯 Your Mission + 🏆 Challenge

## 🎯 Mission — Polish the UI

Apply all four UX improvements to your Student Management System:

- Bootstrap `alert-success` after insert/update
- Bootstrap `alert-danger` after delete
- Empty state message when no records found
- Bootstrap Icons on action buttons (✏️ Edit, 🗑️ Delete)

## 🏆 Challenge

Add alternating row colors using Bootstrap's `table-striped` class and customize the stripe color with a small CSS override to match your app's theme.

## ⭐ Bonus

Display a dynamic counter badge at the top of the table: `"Total Students: 12"` — pulled live from a `SELECT COUNT(*)`

# Bug Hunt — Guess the Result

### Bug #1 — Wrong SQL Keyword

A student used `INSERT` instead of `UPDATE` in the edit form handler. What happens when they save?

### Bug #2 — Missing WHERE Clause

The `UPDATE` query runs without `WHERE id = ?`

Every student's data is now identical. How do you prevent this?

### Bug #3 — Delete Without ID

The delete link passes no ID. `DELETE FROM students` runs without a WHERE clause. The entire table is wiped.

### Bug #4 — No Primary Key

The table was created without a primary key. Edit and Delete have no reliable way to target a single row. Always define `id INT AUTO_INCREMENT PRIMARY KEY`.

# 🚀 Build Sprint — 30 Minutes

This is your final boss. Build a complete, production-ready **Student Management Portal** from scratch — or extend your existing project. **Timer starts now.**

### ✅ Create

Add new student with form validation

### 📋 Read

Display all students in a Bootstrap table

### ✏️ Update

Pre-filled edit form with success alert

### 🗑️ Delete

Confirm dialog + success message

### 🔍 Search

Filter by name and branch

### ⭐ Bonus

Responsive layout + professional icons + record count

# 🎤 Demo Day — Student Showcase

Each team gets exactly **2 minutes** to present their Student Management System to the class. The instructor will ask one technical question per team — so be ready to explain your code, not just show it.

01

### Show Your App Live

Demo the running application in the browser — add, edit, delete, and search a student record live.

02

### Walk Through CRUD

Briefly point to the code for each operation and explain what each block does in plain English.

03

### Biggest Challenge

What was the hardest part to implement? What error did you hit, and how did you solve it?

04

### One Future Improvement

If you had 30 more minutes, what feature would you add next — and why?

# 🧠 Interview Quiz — Think Fast

These are real interview questions asked by tech companies. Answer out loud. No Googling. **Go.**

1

**INSERT vs. UPDATE**

What is the exact difference? When would you use each one in a Student Management System?

2

**The WHERE Clause**

What is the purpose of `WHERE` in a DELETE or UPDATE query? What happens if you omit it?

3

**No Primary Key**

If a table has no primary key, how do you uniquely identify a student to edit or delete? What problems arise?

4

**Confirm Before Delete**

Why is a confirmation dialog best practice? What alternative pattern do production apps use instead of hard-deleting records?

5

**Predict the Output**

What does `SELECT * FROM students WHERE cgpa > 8.0 ORDER BY name ASC` return? Describe the result set.

# 📋 Assignment — Push It Further

Tonight's assignment extends your Student Management System into a more complete, real-world application. **Push your final code to GitHub** before the start of tomorrow's session.

### 📥 Profile Photo Upload

Add a file input to the student form. Store the filename in MySQL and display the photo in the student table using an `<img>` tag.

### 📥 Student Status

Add an `Active / Inactive` status dropdown. Filter the table to show only active students by default, with an option to view all.

### 📥 Course Filter

Add a dropdown filter above the table to show students by department or course. Use a `WHERE branch = ?` query.

### 📥 Multi-Field Search

Extend search to cover name, email, branch, and CGPA range simultaneously using combined `WHERE` conditions.

### 📥 Better Dashboard

Add a stats row at the top: Total Students · Average CGPA · Students per Branch — pulled from SQL aggregate queries.